

Sorting

Table of contents

Ordinamento	1
Ordinamento	1
Stabilità	2
Algoritmo stabile	2
Esempio	3
Algoritmi stabili	3
Algoritmo instabile	3
Algoritmi instabili	4
Perché è importante la stabilità	4

Ordinamento

Ordinamento

In informatica, **l'ordinamento** è il processo con cui si dispone una collezione di dati secondo un criterio definito, ad esempio in ordine crescente o decrescente.

Può riguardare numeri, parole, record o altri elementi.

Esempi:

- 3, 1, 5, 2 $\rightarrow$ 1, 2, 3, 5 (ordine crescente)
- pera, mela, banana $\rightarrow$ banana, mela, pera (ordine alfabetico)

Per illustrare gli algoritmi di ordinamento, per comodità consideriamo un array (o una lista) di interi, ma gli stessi algoritmi si applicano a strutture dati più generali.

L'essenziale è che esista una **relazione d'ordine** tra gli elementi da ordinare.

In c:

L'elemento dell'array può essere una struct. Occorre ordinare l'array secondo un criterio che coinvolge uno o più campi della struct.

Esempio:

```
struct lavoratore{
    float paga_oraria;
    float ore_lavorate;
    // altri campi
}
```

- Ordinare secondo la paga oraria crescente
 - Ordinare secondo lo stipendio crescente ($\text{ore_lavorate} * \text{paga_oraria}$)
-

In Python:

L'elemento della lista può essere un dizionario. Occorre ordinare il dizionario secondo un criterio che coinvolge uno o più chiavi del dizionario.

Esempio:

```
d = { paga_oraria: 26,
      ore_lavorate: 32,
      # altri campi
    }
```

- Ordinare secondo la paga oraria crescente
- Ordinare secondo lo stipendio crescente ($\text{ore_lavorate} * \text{paga_oraria}$)

Stabilità

Un algoritmo di ordinamento può essere **stabile** oppure **instabile**.

Algoritmo stabile

Un algoritmo è **stabile** se mantiene l'ordine originale degli elementi con chiave uguale.

Esempio

Supponiamo di ordinare per voto:

Nome	Voto
Luca	8
Anna	6
Marco	8

Dopo un ordinamento stabile:

Nome	Voto
Anna	6
Luca	8
Marco	8

Luca rimane prima di Marco perché era già così nell'elenco iniziale.

Algoritmi stabili

- Bubble Sort
- Insertion Sort
- Merge Sort
- Counting Sort

Algoritmo instabile

Un algoritmo è **instabile** se può cambiare l'ordine relativo degli elementi con chiave uguale.

Con lo stesso esempio, dopo l'ordinamento potremmo ottenere:

Nome	Voto
Anna	6
Marco	8
Luca	8

Marco e Luca hanno scambiato posizione.

Algoritmi instabili

- Quick Sort
- Selection Sort
- Heap Sort

Perché è importante la stabilità

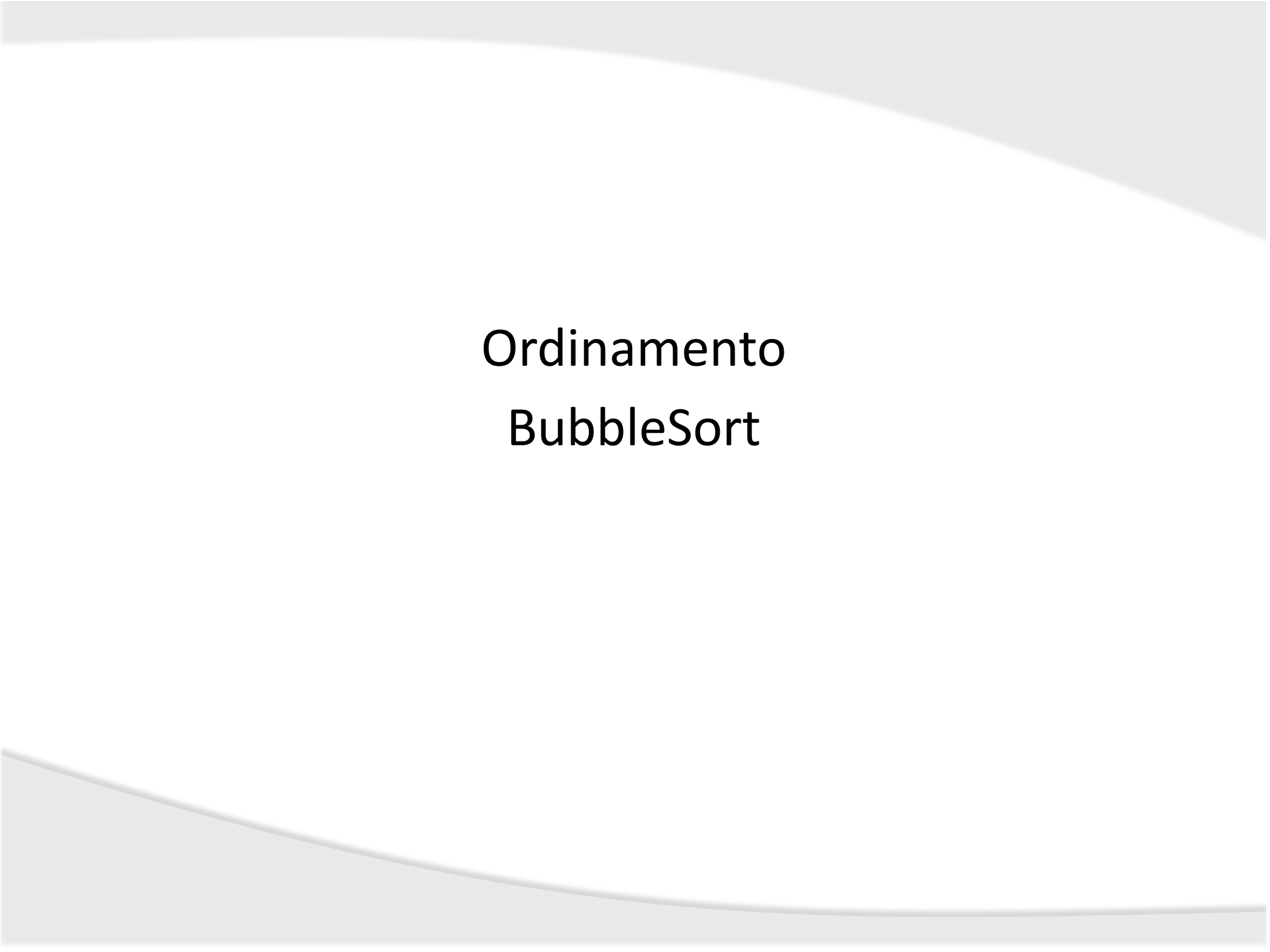
La stabilità è utile quando:

- si ordinano dati con più campi,
- si effettuano ordinamenti successivi,
- si vuole preservare un ordine precedente.

Esempio:

1. ordino per nome,
2. poi ordino per voto.

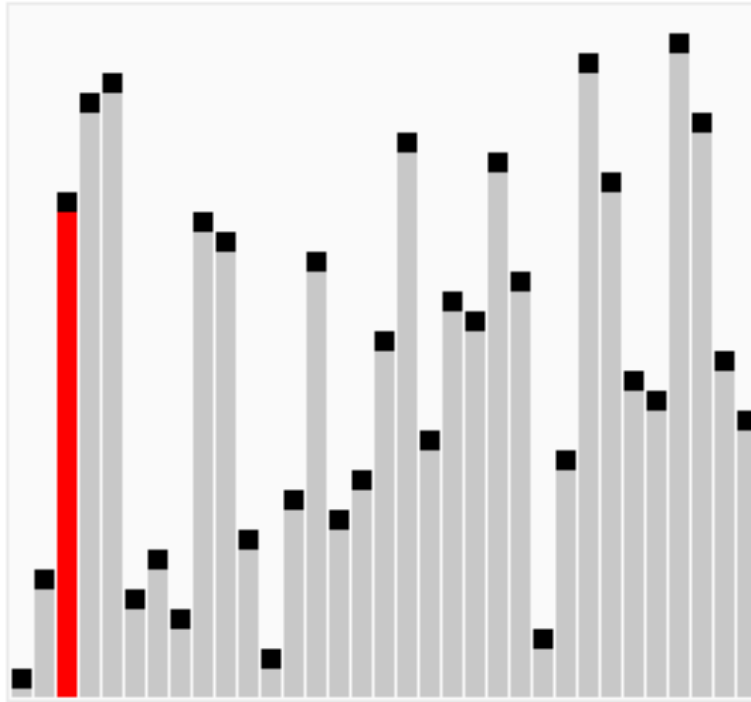
Se il secondo algoritmo è stabile, gli studenti con lo stesso voto resteranno ordinati alfabeticamente.



Ordinamento BubbleSort

Bubble Sort

Ogni coppia di elementi adiacenti viene comparata e invertita di posizione se sono nell'ordine sbagliato.



Fonte immagine: https://it.wikipedia.org/wiki/Bubble_sort

"Because it is not very good relative to other methods and is the one typically stumbled on by naive and untutored programmers, hackers consider it the canonical example of a **naive algorithm**."

<http://www.jargon.net/jargonfile/b/bubblesort.html>

Bubble Sort

6 5 3 1 8 7 2 4

Fonte immagine: https://it.wikipedia.org/wiki/Bubble_sort

Bubble Sort

```
int main() {  
    int vett[6]={9,18,7,15,21,11};  
    int nEl=6;  
    printArray(vett, nEl);  
    bubbleSort_1(vett, nEl);  
    printArray(vett, nEl);  
    getchar();  
    return 0;  
}
```

Bubble Sort

```
void bubbleSort_1(int* v, int n){  
    int tmp;  
    for (int j=0; j<n-1; j++){  
        for (int i=0; i<n-1; i++){  
            if (v[i]>v[i+1]){  
                // SWAP  
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;  
            }  
        }  
    }  
}
```

```
[9 18 7 15 21 11 ]  
iter: 0  
iter: 1  
iter: 2  
iter: 3  
iter: 4  
[7 9 11 15 18 21 ]
```

Quando un intero ciclo interno non ha dato luogo a swap, l'array è ordinato.
Possiamo migliorare l'implementazione.

bubbleSort_1.c

Bubble Sort

Ottimizzazione:

quando non avvengono più swap per un intero ciclo esterno, l'array è ordinato. Posso interrompere la procedura di ordinamento.

Bubble Sort

```
void bubbleSort_2a(int* v, int n){
    int tmp;
    bool continua;
    for (int j=0; j<n-1; j++){
        printf("iter: %d \n",j);
        continua=false;
        for (int i=0; i<n-1; i++){
            if (v[i]>v[i+1]){ // SWAP
                continua=true;
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;
            }
        }
    }
    if (continua==false) break;
```

```
[9 18 7 15 21 11 ]
iter: 0
iter: 1
iter: 2
iter: 3
[7 9 11 15 18 21 ]
```

bubbleSort_2.c

Bubble Sort

Stesso codice, ma implementato mediante do-while

```
void bubbleSort_2b(int* v, int n){  
    int tmp,j=0;  
    bool continua;  
    do {  
        printf("iter: %d \n",j++);  
        continua=false;  
        for (int i=0; i<n-1; i++){  
            if (v[i]>v[i+1]){ // SWAP  
                continua=true;  
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;  
            }  
        }  
    }  
    while (continua);  
}
```

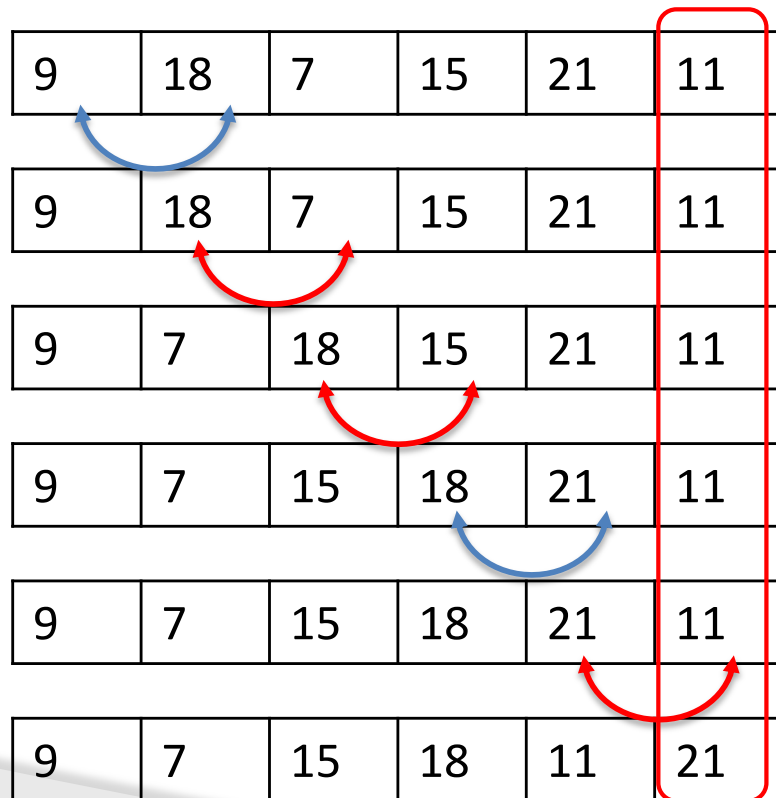
```
[9 18 7 15 21 11 ]  
iter: 0  
iter: 1  
iter: 2  
iter: 3  
[7 9 11 15 18 21 ]
```

bubbleSort_2.c

Bubble Sort

Ottimizzazione: il ciclo **interno** sposta di volta in volta l'elemento più grande verso destra.

Dunque ad ogni iterazione del ciclo **esterno** (j), gli ultimi ($j+1$) elementi del vettore sono ordinati!



Dopo l'iterazione $j=0$,
gli ultimi $j+1=1$ elementi dell'array
sono ordinati.

Dopo l'iterazione $j=1$,
gli ultimi $j+1=2$ elementi dell'array
sono ordinati.

...

Bubble Sort

Ottimizzazione:

ad ogni iterazione del ciclo **esterno** (j), gli ultimi ($j+1$) elementi del vettore sono ordinati.

[cioè sono ordinati gli elementi da $v[n-1-j]$ a $v[n-1]$]

Dunque non è necessario che il **ciclo interno** arrivi fino a ($i < n-1$)

Posso fermare il ciclo interno a ($i < n-1-j$)

Bubble Sort

```
void bubbleSort_3a(int* v, int n){
    int tmp;
    bool continua;
    for (int j=0; j<n-1; j++){
        continua=false;
        for (int i=0; i<n-1-j; i++){
            if (v[i]>v[i+1]){ // SWAP
                continua=true;
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;
            }
        }
        if (continua==false) break;
    }
}
```

bubbleSort_3.c

Bubble Sort

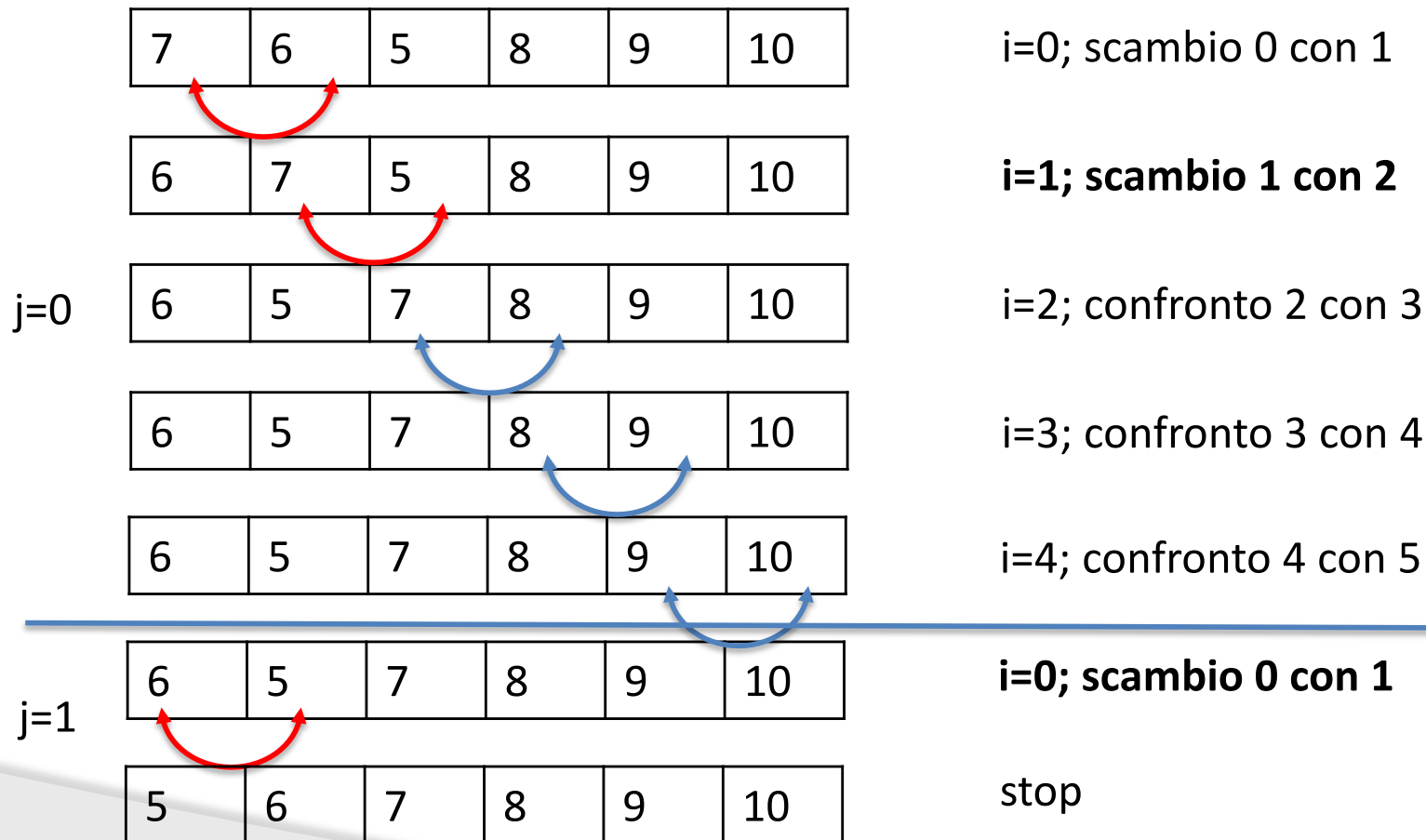
Stesso codice, ma implementato mediante do-while

```
void bubbleSort_3b(int* v, int n){
    int tmp,j=0;
    bool continua;
    do {
        continua=false;
        for (int i=0; i<n-1; i++){
            if (v[i]>v[i+1]){ // SWAP
                continua=true;
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;
            } // end IF
        } // end FOR
        n=n-1;
    }
    while (continua && n>1);
```

bubbleSort_3.c

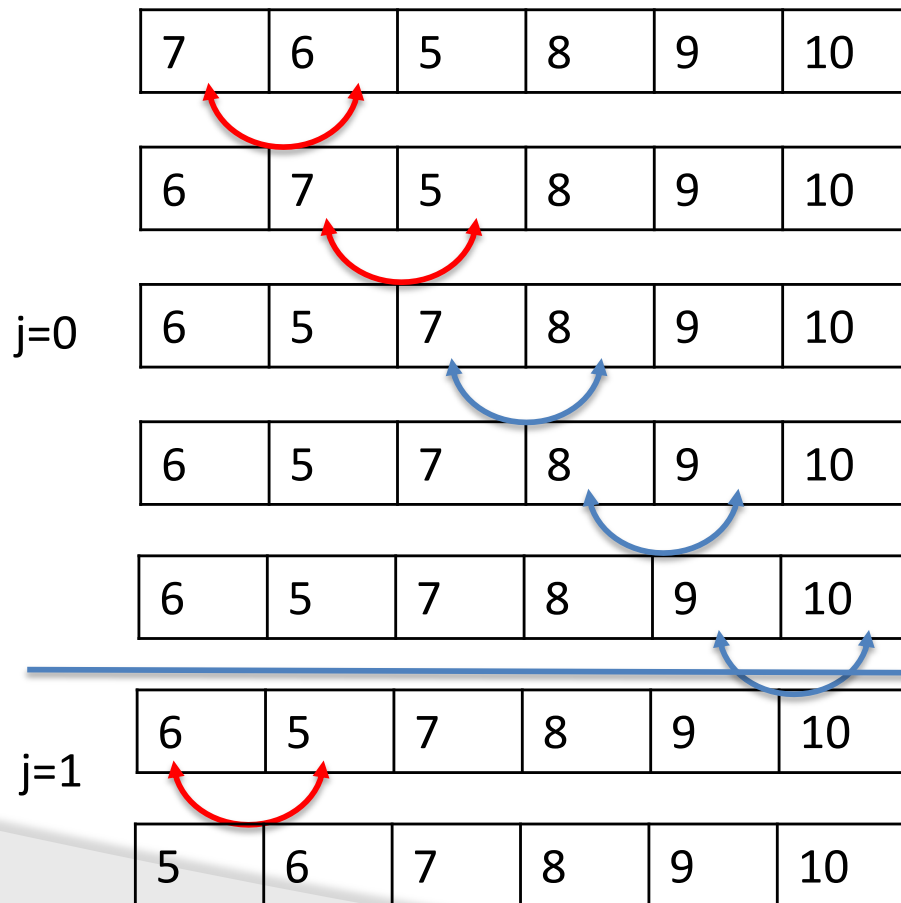
Bubble Sort

Se il **ciclo interno** effettua l'ultimo spostamento fra gli indici i e $i+1$, non è più necessario controllare gli elementi da i in poi.



Bubble Sort

Se il **ciclo interno** effettua l'ultimo spostamento fra gli indici i e $i+1$, non è più necessario controllare gli elementi da i in poi.



$i=0$; scambio 0 con 1

$i=1$; scambio 1 con 2

non devo più controllare $i=1$

$i=0$ è l'ultimo valore

da controllare al prossimo ciclo

pongo $n=i+1$ (ultimo i utilizzato)

perché la condizione del for è

$i < n-1$

$i=0$; scambio 0 con 1

stop

Bubble Sort

```
void bubbleSort_4(int* v, int n) {  
    int tmp, j=0, p=n;  
    bool continua;  
    do {  
        continua=false;  
        for (int i=0; i<n-1; i++) {  
            if (v[i]>v[i+1]) { // SWAP  
                continua=true;  
                p=i+1;  
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;  
            } // end IF  
        } // end FOR  
        n=p;  
    }  
    while (continua && n>1);  
}
```

L'ultimo scambio coinvolge i, i+1
Pongo quindi n= (i+1)
[usando p come appoggio]
in modo che al ciclo successivo
l'ultima coppia confrontata sia (i-1, i)

bubbleSort_4.c

Bubble Sort

Complessità computazionale

Bubble Sort

```
void bubbleSort_1(int* v, int n){  
    int tmp;  
    for (int j=0; j<n-1; j++){  
        for (int i=0; i<n-1; i++){  
            if (v[i]>v[i+1]){  
                // SWAP  
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;  
            }  
        }  
    }  
}
```

$O(n^2)$

Bubble Sort – Caso Migliore

CASO MIGLIORE
VETTORE GIA' ORDINATO

```
void bubbleSort_4(int* v, int n){
    int tmp,j=0, p=n;
    bool continua;
    do {
        continua=false;
        for (int i=0; i<n-1; i++){
            if (v[i]>v[i+1]){ // SWAP
                continua=true;
                p=i+1;
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;
            } // end IF
        } // end FOR
        n=p;
    }
    while (continua && n>1);
}
```

Assegnamento esterno al ciclo: 2

DO: numero di test: 1; iterazioni: 1

FOR: numero di test: n; iterazioni: n-1

Computazioni interne al ciclo: 1 test per ogni iterazione

$$f(n) = 2 + 1 + n + 1 \cdot (n - 1) = 2n + 2$$

$$O(n)$$

Bubble Sort – Caso Peggior

```
void bubbleSort_4(int* v, int n){
    int tmp, j=0, p=n;
    bool continua;
    do {
        continua=false;
        for (int i=0; i<n-1; i++){
            if (v[i]>v[i+1]){ // SWAP
                continua=true;
                p=i+1;
                tmp=v[i]; v[i]=v[i+1]; v[i+1]=tmp;
            } // end IF
        } // end FOR
        n=p;
    }
    while (continua && n>1);
}
```

CASO PEGGIORE
VETTORE IN ORDINE
INVERSO

Siamo nel caso 'Algoritmo che cicla su input eliminando un elemento per volta'

$$f(n) = f(n - 1) + n = n \cdot \frac{n + 1}{2}$$

$O(n^2)$

Ricorrenza Fondamentale

Algoritmo che cicla su input **eliminando** un elemento per volta
(esempio: bubble sort, selection sort)

$$\begin{aligned}C_N &= C_{N-1} + N; & N \geq 2; C_1 &= 1 \\C_N &= \\&= C_{N-1} + N = \\&= C_{N-2} + (N - 1) + N = \\&= C_{N-3} + (N - 2) + (N - 1) + N = \\&\dots \\&= 1 + 2 + \dots + (N - 2) + (N - 1) + N = \\&= \frac{N(N+1)}{2} \cong \frac{N^2}{2}\end{aligned}$$

Bubble Sort – Caso Medio

Il calcolo non è immediato, ma possiamo fare le seguenti considerazioni

- Cosa significa 'caso medio'? Dobbiamo formulare ipotesi circa la distribuzione da cui campioniamo i vettori
- Se i vettori non sono precedentemente ordinati, abbiamo comunque due cicli annidati.
- Le costanti in $f(n)$ sono legate a quanto 'precocemente' l'algoritmo esce dai cicli
- Ma il fatto che ci siano due cicli annidati implica che la complessità sia $O(n^2)$, come nel caso peggiore.

Cenni al calcolo esatto:

<https://math.stackexchange.com/questions/453686/what-is-the-expected-number-of-swaps-performed-by-bubblesort>

Esercizi

- Illustrate l'operazione di BubbleSort su un array dato
- Calcolare la complessità $f(n)$ utilizzando come input un array dato. Verificare la correttezza di $f(n)$ contando le istruzioni durante l'esecuzione.
- Ipotesizzare che lo spostamento di un valore dell'array (dunque $e_l = v[i]$, $v[j+1] = v[j]$, ecc) abbia costo c rispetto alle altre istruzioni elementari. L'ipotesi ha senso perché gli elementi da spostare potrebbero essere strutture. Calcolare $f(n)$.

Esercizi

- Applicare l'algoritmo ad un array DI RECORD
- Applicare l'algoritmo ad un array di PUNTATORI a RECORD

(l'ordinamento ha come oggetto uno dei campi del record)

Contate i passi base per i seguenti casi specifici, con $n=8$

- vettore ordinato in ordine inverso
- $V=[6\ 1\ 4\ 5\ 3\ 2\ 3\ 7]$
- Prima parte del vettore ordinata, seconda parte ordinata in ordine inverso.
- Usare una lista concatenata al posto di un vettore statico

Esercizi

- Modificare BubbleSort in modo che disponga gli elementi in ordine non crescente
- Modificare BubbleSort in modo che l'ordinamento possa essere crescente o decrescente. È possibile evitare di introdurre un ulteriore IF all'interno dell'algoritmo?

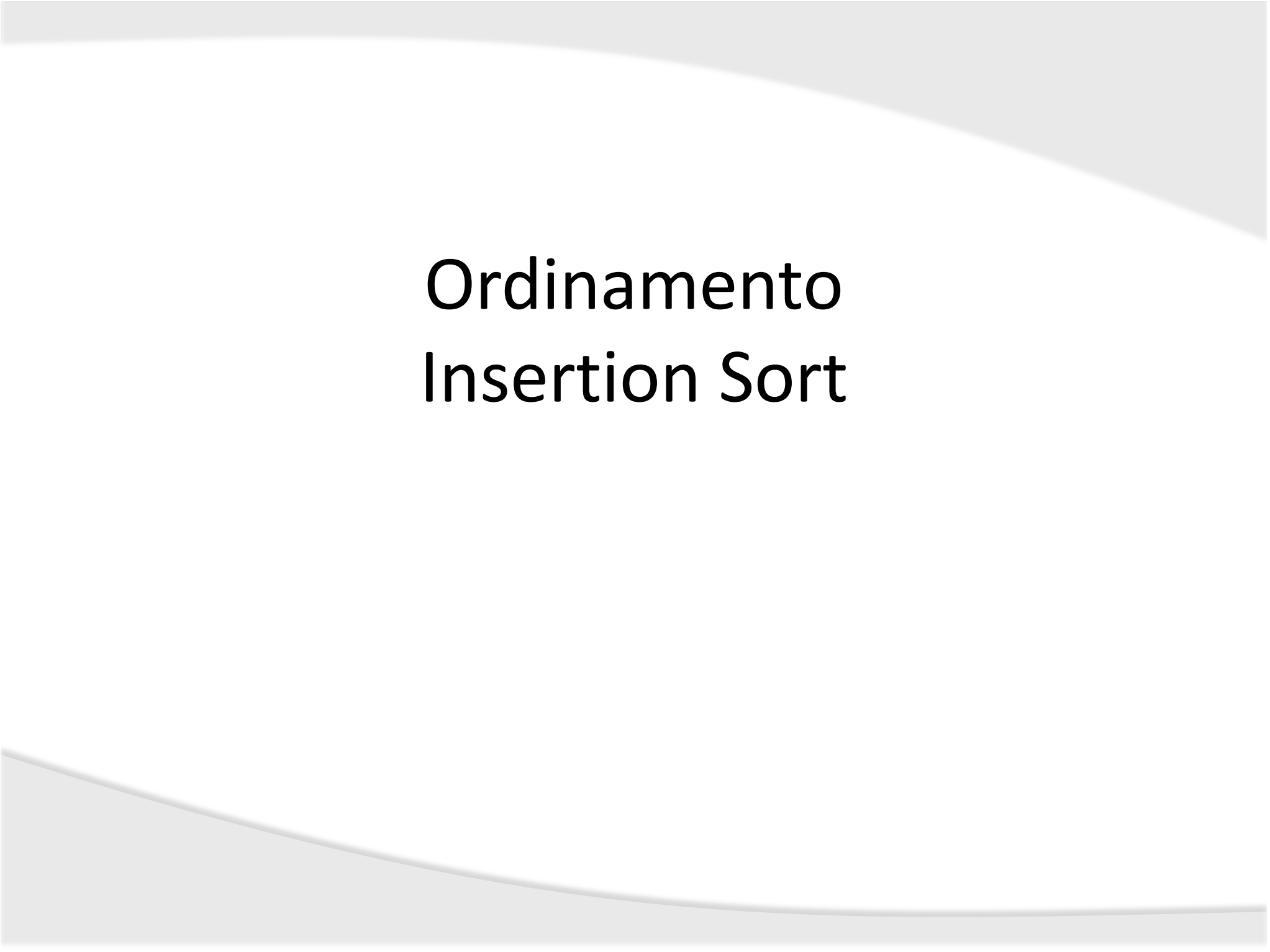
Generalizzare bubble sort in modo che

- sia possibile ordinare in modo crescente o decrescente.
- in caso di array di strutture, sia possibile scegliere il campo (o i campi) da considerare per l'ordinamento

Main e funzione di sorting devono essere disaccoppiate!

Si tratta di una semplice applicazione dello **strategy design pattern**. Lo stesso meccanismo si può applicare a tutti gli algoritmi di ordinamento basati su confronto

(vedi soluzione in ``bubblesort_strategy``)



Ordinamento Insertion Sort

Insertion Sort

Consideriamo un vettore di interi e dividiamolo in

- parte **ordinata** (all'inizio un solo elemento) e
- parte **non ordinata**

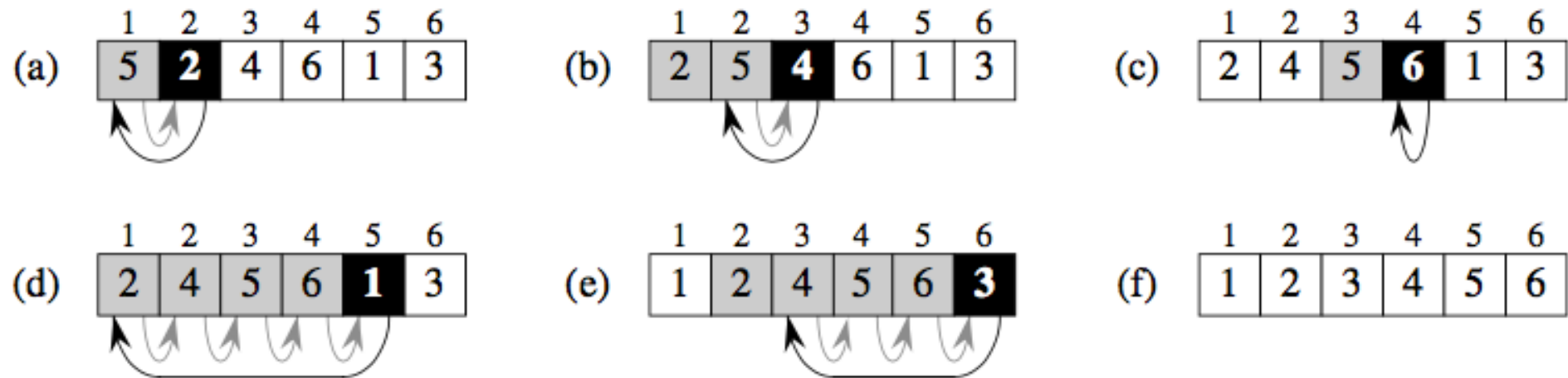
6 5 3 1 8 7 2 4

Ad ogni iterazione, prendo un elemento dalla parte non ordinata e lo inserisco nella parte ordinata

- meno efficiente di algoritmi più avanzati
- semplice da implementare
- efficiente per vettori quasi ordinati.

Immagine animata in
https://en.wikipedia.org/wiki/Insertion_sort

Insertion Sort



Ad ogni iterazione, il rettangolo nero contiene il valore estratto, che viene confrontato con la porzione di vettore alla sua sinistra, già ordinata.

Il valore estratto viene spostato in modo da mantenere ordinata la porzione di vettore a sinistra, shiftando di conseguenza gli elementi.

Insertion Sort

```
void InsertionSort(int* v, int n) {
    int el, j;
    for (int i = 1; i < n; i++) {
        el = v[i];
        // la porzione di vettore v[0] ... v[i-1]
        // è già ordinata.
        // vi inserisco el, shiftando gli altri valori
        j = i - 1;

        while (j >= 0 && v[j] > el) {
            // NB Short Circuit Evaluation
            v[j + 1] = v[j]; // SHIFT
            j--;
        }
        v[j + 1] = el;
    }
}
```

Insertion Sort

```
void InsertionSort(int* v, int n) {  
    int el, j;  
    for (int i = 1; i < n; i++) {  
        el = v[i];  
        j = i - 1;  
  
        while (j >= 0 && v[j] > el) {  
            v[j + 1] = v[j]; // SHIFT  
            j--;  
        }  
        v[j + 1] = el;  
    }  
}
```

Numero esecuzioni

n

n-1

n-1

$\sum_{i=1}^{n-1} t_i$

$\sum_{i=1}^{n-1} (t_i - 1)$

$\sum_{i=1}^{n-1} (t_i - 1)$

n-1

Insertion Sort

```
void InsertionSort(int* v, int n) {  
    int el, j;  
    for (int i = 1; i < n; i++) {  
        el = v[i];  
        j = i - 1;  
  
        while (j >= 0 && v[j] > el) {  
            v[j + 1] = v[j]; // SHIFT  
            j--;  
        }  
        v[j + 1] = el;  
    }  
}
```

Numero esecuzioni

n

n-1

n-1

$\sum_{i=1}^{n-1} t_i$

$\sum_{i=1}^{n-1} (t_i - 1)$

$\sum_{i=1}^{n-1} (t_i - 1)$

n-1

Il tempo di esecuzione dipende dal *valore* dell'input, non solo dalla dimensione.

$$f(n) = n + 2(n - 1) + \sum_{i=1}^{n-1} t_i + 2 \sum_{i=1}^{n-1} (t_i - 1) + (n - 1) = 2n - 1 + 3 \sum_{i=1}^{n-1} t_i$$

Insertion Sort

```
void InsertionSort(int* v, int n) {  
    int el, j;  
    for (int i = 1; i < n; i++) {  
        el = v[i];  
        j = i - 1;  
  
        while (j >= 0 && v[j] > el) {  
            v[j + 1] = v[j]; // SHIFT  
            j--;  
        }  
        v[j + 1] = el;  
    }  
}
```

Caso migliore:
vettore già ordinato
(crescente)

$el \geq v[j]$ per ogni i
quando $j = i - 1$

test ciclo while
sempre FALSE

$t_i = 1$

$$f(n) = 2n - 1 + 3 \sum_{i=1}^{n-1} t_i = 5n - 4$$

$O(n)$

Insertion Sort

```
void InsertionSort(int* v, int n) {
    int el, j;
    for (int i = 1; i < n; i++) {
        el = v[i];
        j = i - 1;

        while (j >= 0 && v[j] > el) {

            v[j + 1] = v[j]; // SHIFT
            j--;
        }
        v[j + 1] = el;
    }
}
```

Caso peggiore:
vettore ordinato in
senso inverso
(decrescente)

$el < v[j]$

ciclo while si
interrompe quando
 $j < 0$

$t_i = i + 1$

$$f(n) = 2n - 1 + 3 \sum_{i=1}^{n-1} t_i = \frac{3}{2}n^2 + \frac{7}{2}n - 4$$

$O(n^2)$

Considerazioni su complessità

Caso peggiore:

Limite superiore per qualsiasi input (abbiamo la garanzia che non verrà superato).

Il caso peggiore si verifica spesso

Considerazioni su complessità

Caso medio:

Molto difficile da calcolare (a volte impossibile).

Il caso medio spesso è brutto quanto il caso peggiore.

Es. quando l'algoritmo deve determinare dove inserire l'elemento $a[i]$ nel sottoarray $a[0 \dots i-1]$, in media metà degli elementi del sottoarray saranno più grandi di $a[i]$. Ci aspettiamo $t_i \cong \frac{i}{2}$ e quindi $O(n^2)$

Esercizi

- Illustrate l'operazione di InsertionSort su un array dato
- Calcolare la complessità $f(n)$ utilizzando come input un array dato. Verificare la correttezza di $f(n)$ contando le istruzioni durante l'esecuzione.
- Ipotesizzare che lo spostamento di un valore dell'array (dunque $e1=v[i]$, $v[j+1]=v[j]$, ecc) abbia costo doppio rispetto alle altre istruzioni elementari. L'ipotesi ha senso perché gli elementi da spostare potrebbero essere strutture. Calcolare $f(n)$.

Esercizi

- Modificare InsertionSort in modo che disponga gli elementi in ordine non crescente
- Modificare InsertionSort in modo che ammetta una certa tolleranza nell'ordinamento. Calcolare $f(n)$.
L'algoritmo modificato è più rapido di quello originale?
Che problemi da?

Esercizi

- Implementare insertion sort in modo ricorsivo e calcolare complessità computazionale.

Esercizi

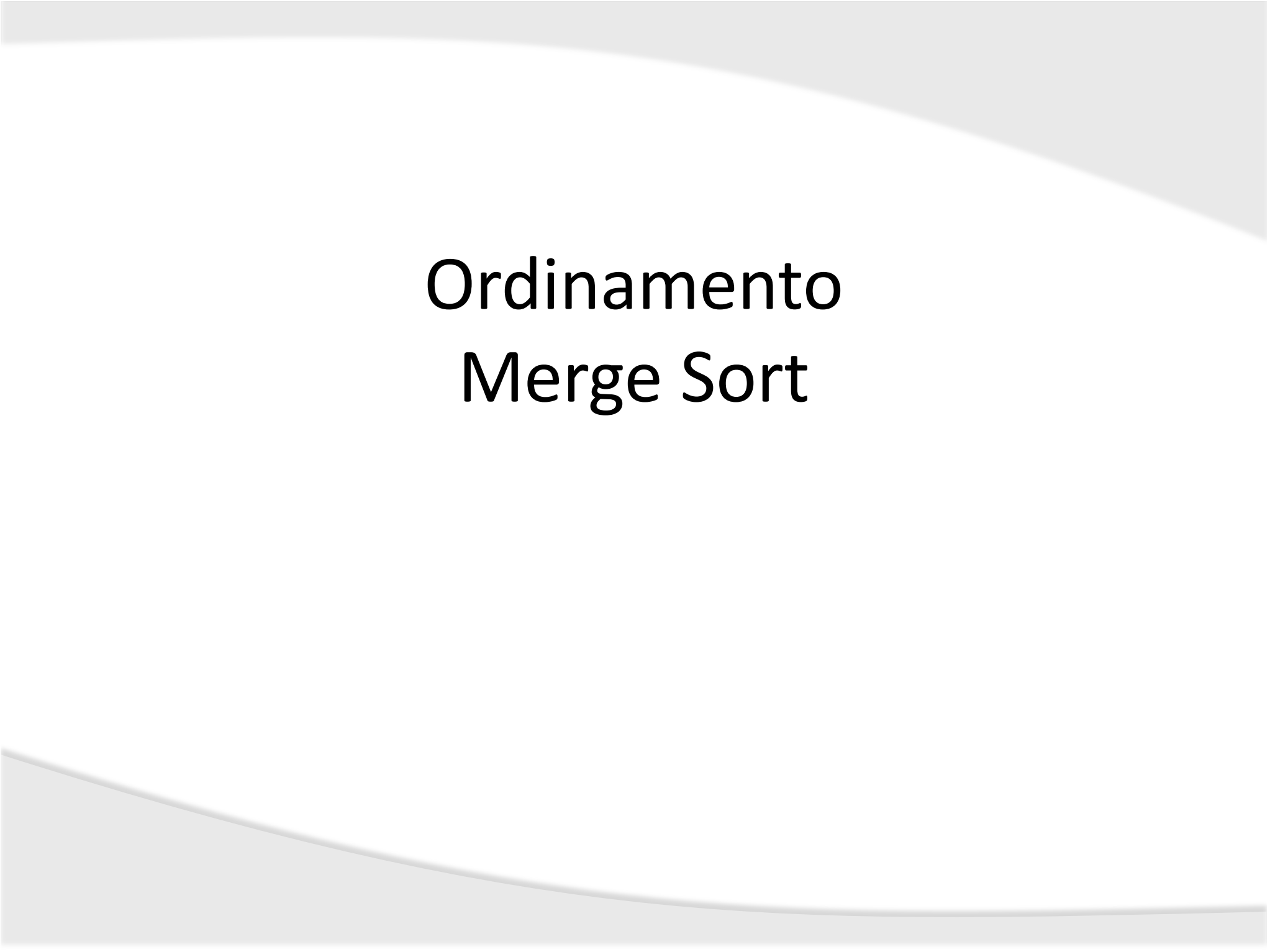
Algoritmo SelectionSort:

- si trovi l'elemento più piccolo di **a**, e si scambi con **a[0]**
- si trovi l'elementi più piccolo di **a[1... n-1]**, e si scambi con **a[1]**
- ...
- si trovi l'elementi più piccolo di **a[j... n-1]**, e si scambi con **a[j]**

Implementare l'algoritmo.

Trovare complessità computazionale.

Svolgere gli esercizi proposti per InsertionSort



Ordinamento Merge Sort

Merge Sort

InsertionSort usa un approccio **incrementale**: dopo aver ordinato $v[0 \dots i-1]$, inserisce $v[i]$ nel punto giusto, ottenendo $v[0 \dots i]$ ordinato.

Altro approccio è quello **divide et impera**:

- suddividiamo il problema in sottoproblemi simili al problema originale ma di dimensioni più piccole
- risolviamo i sottoproblemi in modo ricorsivo
- combiniamo le soluzioni

Un tipico esempio è il **Merge Sort**

Merge Sort

Divide: divide la sequenza degli n elementi da ordinare in due sottosequenze di $n/2$ elementi ciascuna.

Impera: ordina le due sottosequenze in modo ricorsivo utilizzando l'algoritmo Merge Sort.

Combina: fonde le due sottosequenze ordinate per generare la sequenza ordinata.

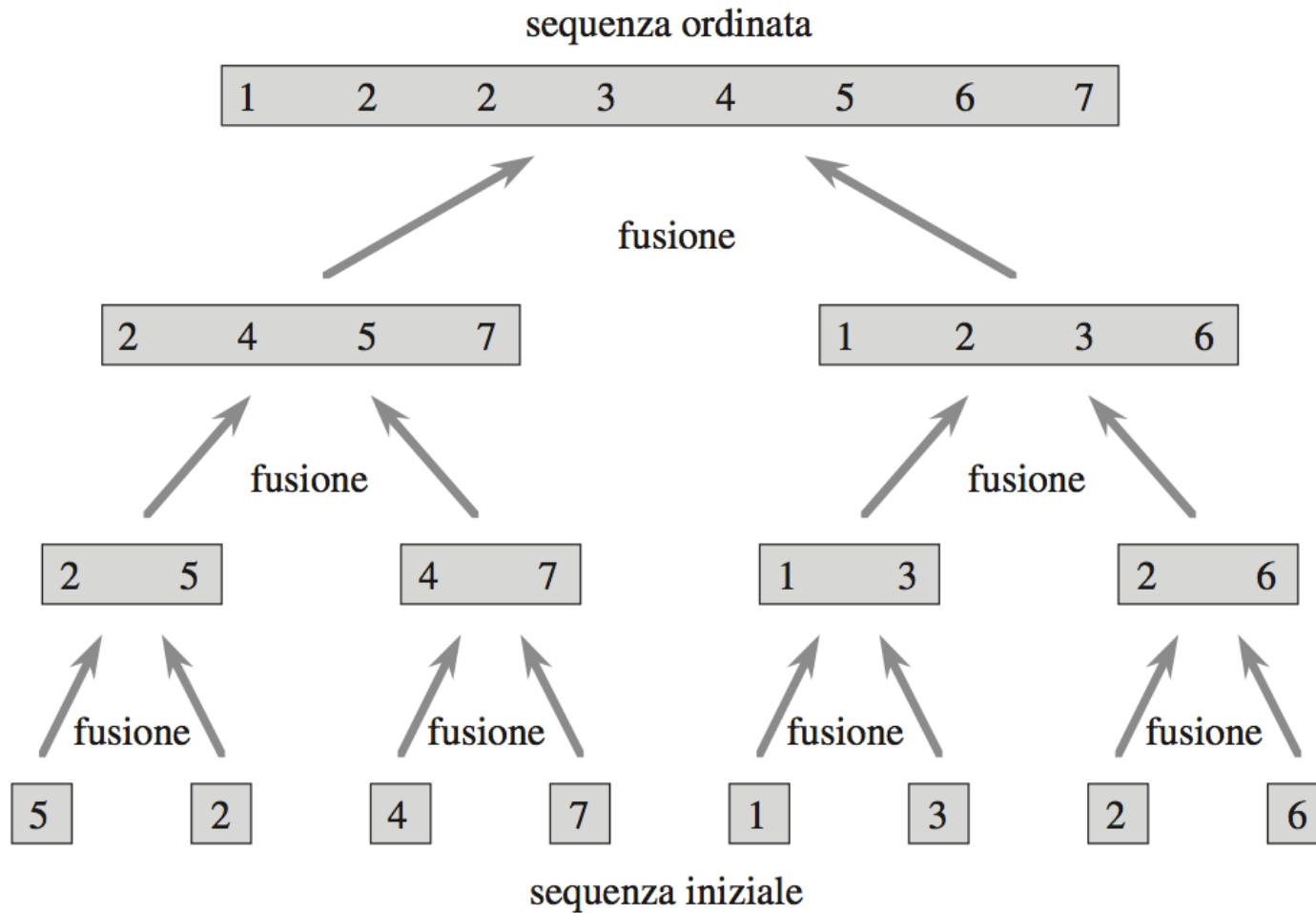
Quando la sequenza da ordinare ha lunghezza 1, la ricorsione si interrompe (ogni sequenza di lunghezza 1 è già ordinata)

Merge Sort

5 2 4 7 1 3 2 6

sequenza iniziale

Merge Sort

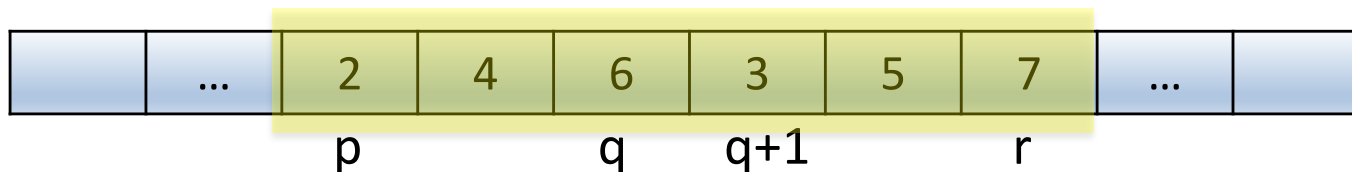


Merge Sort

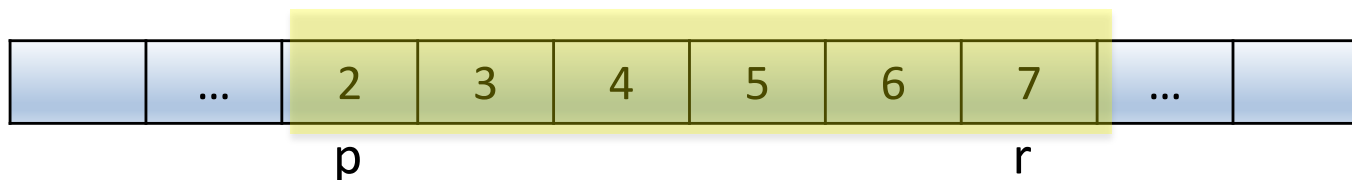
Il passo chiave dell'algoritmo è nella procedura MERGE

INPUT: array $\mathbf{v}$, interi p, q, r :

$\mathbf{v}[p \dots q]$ e $\mathbf{v}[q+1 \dots r]$ sono ordinati



OUTPUT: array $\mathbf{v}$, con $\mathbf{v}[p \dots r]$ ordinato



MERGE

Dati due vettori ordinati, fonderli in un unico vettore
(ordinato)

MERGE

Dati due vettori ordinati, fonderli in un unico vettore (ordinato)

Step 1

Scorrere i 2 vettori $v1$, $v2$

Inserire in $v3$ l'elemento più piccolo fra $v1$ e $v2$

il ciclo termina quando ho esplorato completamente uno dei due vettori

Step 2

ricopio in $v3$ il vettore non completamente esaminato.

MERGE

```
int main() {  
    int nEl1=6, nEl2=9, nEl3=15;  
    int vett1[6]={1,3,5,7,9,11};  
    int vett2[9]={2,4,6,8,10,12,13,14,15};  
    int vett3[15];  
    merge(vett1,vett2, vett3, nEl1, nEl2);  
    printArray(vett3, nEl3);  
}
```

MERGE

```
void merge(int* v1, int *v2, int *v3, int n1, int n2){  
    int i1=0,i2=0,i3=0 ;  
    // STEP 1  
    do {  
        if (v1[i1]<=v2[i2]){  
            v3[i3]=v1[i1];  
            i3++; i1++;  
        }  
        else{  
            v3[i3]=v2[i2];  
            i3++; i2++;  
        }  
    }  
    while (i1<n1 && i2<n2);  
}
```

MERGE

// STEP 2

```
while(i1<n1){  
    v3[i3]=v1[i1];  
    i3++; i1++;  
}  
while(i2<n2){  
    v3[i3]=v2[i2];  
    i3++; i2++;  
}
```

MERGE

Posso implementare in modo che operi avendo come ingresso un solo array v

```
void merge(int* v, int p, int q, int r) {  
    int i;  
    int i_left, i_right;  
    int len_left=q-p+1;  
    int len_right=r-q;  
    int v_left[len_left];  
    int v_right[len_right];  
  
    for (i=0;i<len_left;i++) {  
        v_left[i]=v[p+i];  
    }  
    for (i=0;i<len_right;i++) {  
        v_right[i]=v[q+1+i];  
    }  
}
```

Creo i due vettori **left** e **right** che contengono i sottovettori ordinati

...

MERGE

Unisco **left** e **right** , mantenendo l'ordinamento, nel sottovettore **v[q ... r]**

```
i_left=0;
i_right=0;
i=p;

do {
    if (v_left[i_left]<=v_right[i_right]) {
        v[i]=v_left[i_left];
        i++; i_left++;
    }
    else{
        v[i]=v_right[i_right];
        i++; i_right++;
    }
}
while (i_left<len_left && i_right<len_right);
...
```

MERGE

```
while (i_left < len_left) {  
    v[i] = v_left[i_left];  
    i++; i_left++;  
}  
while (i_right < len_right) {  
    v[i] = v_right[i_right];  
    i++; i_right++;  
}  
}
```

Unisco **left** e **right** , mantenendo
l'ordinamento, nel sottovettore **v[q ... r]**

Complessità computazionale (asintotica)
 $O(n)$

[Codice **merge.c**]

Merge Sort

Ora abbiamo $\text{merge}(v, p, q, r)$ che unisce i sottoinsiemi individuati dagli indici p, q e $q+1, r$.

Possiamo realizzare una procedura

$\text{merge_sort}(a, p, r)$ che ordina gli elementi nel sottoarray $a[p, r]$, in questo modo:

- se $p=r$, il sottoarray ha un solo elemento, ed è ordinato.

- altrimenti, si calcola un **indice q** che separa

$a[p \dots q] \rightarrow \lfloor m/2 \rfloor$ elementi

$a[q+1 \dots r] \rightarrow \lfloor m/2 \rfloor$ elementi

e richiama se stessa.

Merge Sort

Ora abbiamo $\text{merge}(v, p, q, r)$ che unisce i sottoinsiemi individuati dagli indici p, q e $q+1, r$.

Possiamo realizzare una procedura

$\text{merge_sort}(a, p, r)$ che ordina gli elementi nel sottoarray $a[p, r]$, in questo modo:

- se $p=r$, il sottoarray ha un solo elemento, ed è ordinato.
- altrimenti, si calcola un indice q che separa

$a[p \dots q] \rightarrow \lceil m/2 \rceil$ elementi

$a[q+1 \dots r] \rightarrow \lfloor m/2 \rfloor$ elementi

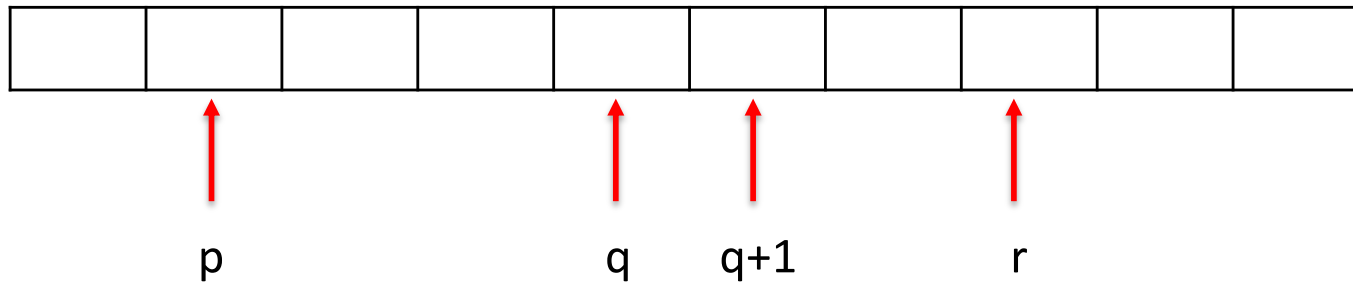
$m/2$ arrotondato all'intero superiore

$m/2$ arrotondato all'intero inferiore

e richiama se stessa.

Merge Sort

```
void merge_sort(int* v, int p, int r) {  
    int q;  
  
    if (p < r) {  
  
        q = (p + r) / 2; // DIVISIONE FRA INTERI  
        merge_sort(v, p, q);  
        merge_sort(v, q + 1, r);  
        merge(v, p, q, r);  
    }  
}
```

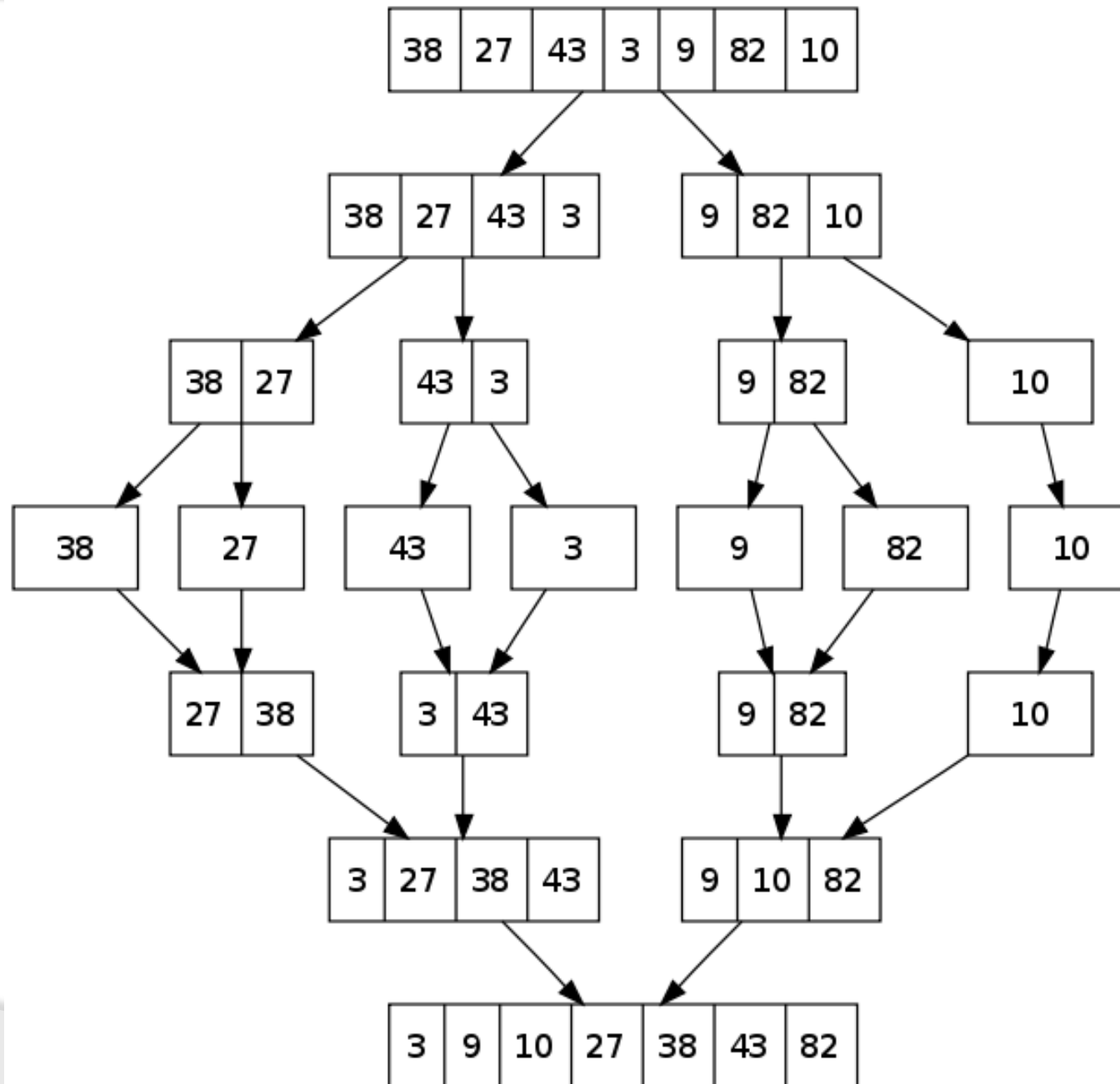


Merge Sort

Infine chiamo **merge_sort** passando l'intero vettore.

```
int main() {  
    int nEl=7;  
    int vett[7]={38, 27, 43, 3, 9, 82, 10};  
    printArray(vett, nEl);  
  
    merge_sort(vett, 0, nEl-1);  
  
    printArray(vett, nEl);  
  
    getchar();  
    return 0;  
}
```

Merge Sort



Merge Sort

```
void merge_sort(int* v, int p, int r) {  
    int q;  
  
    if (p < r) {  
  
        q = (p + r) / 2; // DIVISIONE FRA INTERI  
        merge_sort(v, p, q);  
        merge_sort(v, q + 1, r);  
        merge(v, p, q, r);  
    }  
}
```

Merge Sort

Suggerimenti

1) Definite un vettore di 5 elementi e svolgete l'algoritmo con carta e penna seguendo il flusso del programma

- la prima chiamata a *merge_sort* divide in due il vettore *v*
 - richiama se stessa passando la prima metà di *v*
 - richiama se stessa passando la seconda metà di *v*
 - solo alla fine viene chiamata la *merge*.

2) con lo stesso vettore, utilizzate l'opzione DEBUG di CLION (o altro ambiente) ed eseguite passo-passo il codice. Confrontate i valori in memoria con i valori che avete ricavato con carta e penna. In alternativa potete utilizzare lo strumento online <https://pythontutor.com/c.html#mode=edit> (solo per programmi piccoli)

Merge Sort - Complessità

Analisi dell'algoritmo

Scriviamo una **equazione di ricorrenza** che esprime il tempo di esecuzione totale per un problema di dimensione n in funzione dei tempi di esecuzione per input più piccoli.

Merge Sort - Complessità

Dalle **Ricorrenze Fondamentali** sappiamo che

Formula generale divide et impera

A)

$$C_N = 2 C_{N/2} + 1; N \geq 2; C_1 = 0$$

$$C_N \cong 2N$$

B)

$$C_N = 2 C_{N/2} + N; N \geq 2; C_1 = 0$$

$$C_N = N \log_2 N$$

```
{ ...  
  merge_sort(v, p, q);  
  merge_sort(v, q+1, r);  
  merge(v, p, q, r);  
}
```

Controllando il nucleo
fondamentale dell'algoritmo
vediamo che abbiamo

due operazioni di costo $C_{N/2}$
(merge_sort)

e una operazione $O(N)$ (merge)

Dunque siamo nel caso B

Merge Sort - Complessità

Rivediamo i risultati sulla complessità computazionale con un livello di dettaglio leggermente maggiore (la sostanza non cambia)

Merge Sort - Complessità

$T(n)$ è il tempo di esecuzione per un input n

Se n è piccolo, la soluzione è a tempo costante.

Se n non è piccolo,

dividiamo il problema in a sottoproblemi
di dimensione n/b

Inoltre occorre considerare

- tempo $Div(n)$ per dividere i sottoproblemi (nel caso di mergesort, è il tempo costante necessario a dividere il vettore)
- tempo $Comb(n)$ per combinare le soluzioni (nel caso di mergesort, è il tempo $O(n)$ necessario alla *merge*)

Merge Sort - Complessità

$T(n)$ è il tempo di esecuzione per un input n

Se n è piccolo, la soluzione è a tempo costante.

Se n non è piccolo,

dividiamo il problema in a sottoproblemi
di dimensione n/b

Inoltre occorre considerare

tempo $Div(n)$ per dividere i sottoproblemi

tempo $Comb(n)$ per combinare le soluzioni

$$T(n) = \begin{cases} c & \text{per } n \text{ piccolo} \\ a T\left(\frac{n}{b}\right) + Div(n) + Comb(n) & \text{per } n \text{ grande} \end{cases}$$

Merge Sort - Complessità

Nel caso specifico del MERGE SORT:

$$T(n) = \begin{cases} c & \text{per } n \text{ piccolo} \\ a T\left(\frac{n}{b}\right) + Div(n) + Comb(n) & \text{per } n \text{ grande} \end{cases}$$

Ogni passo genera **due** sottoproblemi ($a=2$)
di dimensione $\frac{1}{2}$ del problema originale ($b=2$)

Merge Sort - Complessità

$$T(n) = \begin{cases} c & \text{per } n \text{ piccolo} \\ 2 T\left(\frac{n}{2}\right) + Div(n) + Comb(n) & \text{per } n \text{ grande} \end{cases}$$

Se $n=1$, mergeSort impiega un tempo costante

Se $n>1$ devo

dividere: calcolo il centro dell'array. $Div(n)$ = tempo costante

risolvere i due sottoproblemi: $2 T\left(\frac{n}{2}\right)$

combinare: (merge) $Comb(n) = O(n)$

Merge Sort - Complessità

$$T(n) = \begin{cases} c & n = 1 \\ 2 T\left(\frac{n}{2}\right) + Div(n) + Comb(n) & n > 1 \end{cases}$$

Se $n=1$, mergeSort impiega un tempo costante

Se $n>1$ devo

dividere: calcolo il centro dell'array. $Div(n)$ = tempo costante

risolvere i due sottoproblemi: $2 T\left(\frac{n}{2}\right)$

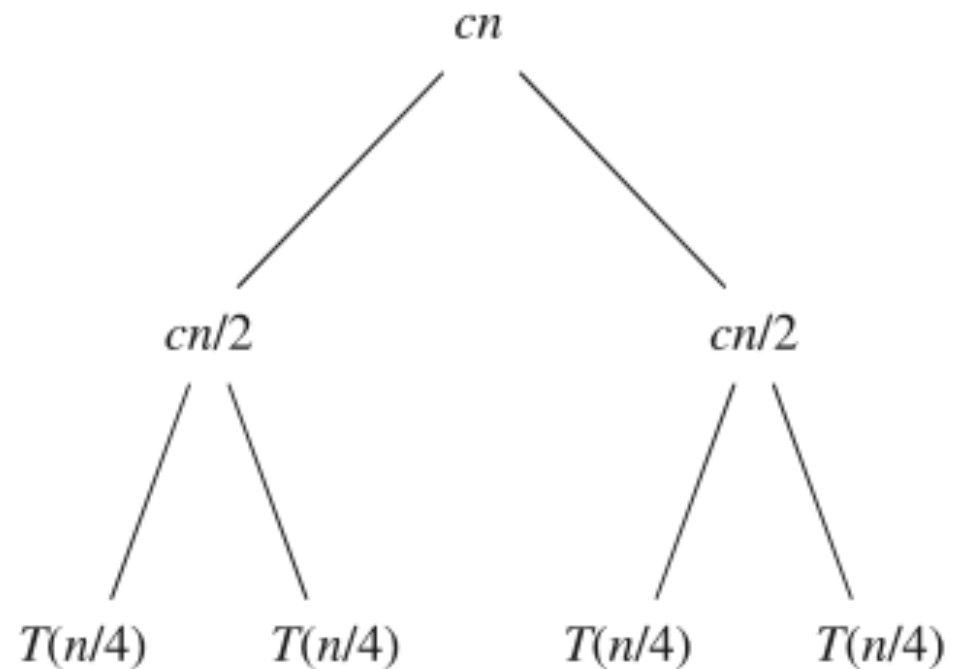
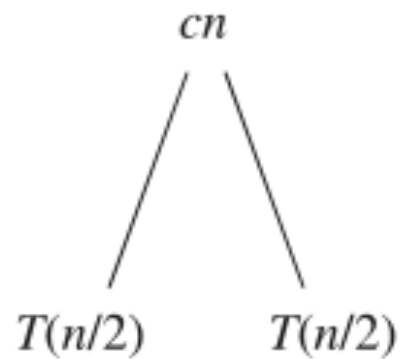
combinare: (merge) $Comb(n) = O(n)$

$$T(n) = \begin{cases} c & n = 1 \\ 2 T\left(\frac{n}{2}\right) + O(n) & n > 1 \end{cases}$$

Merge Sort - Complessità

$$T(n) = \begin{cases} c & n = 1 \\ 2 T\left(\frac{n}{2}\right) + c \cdot n & n > 1 \end{cases}$$

$T(n)$



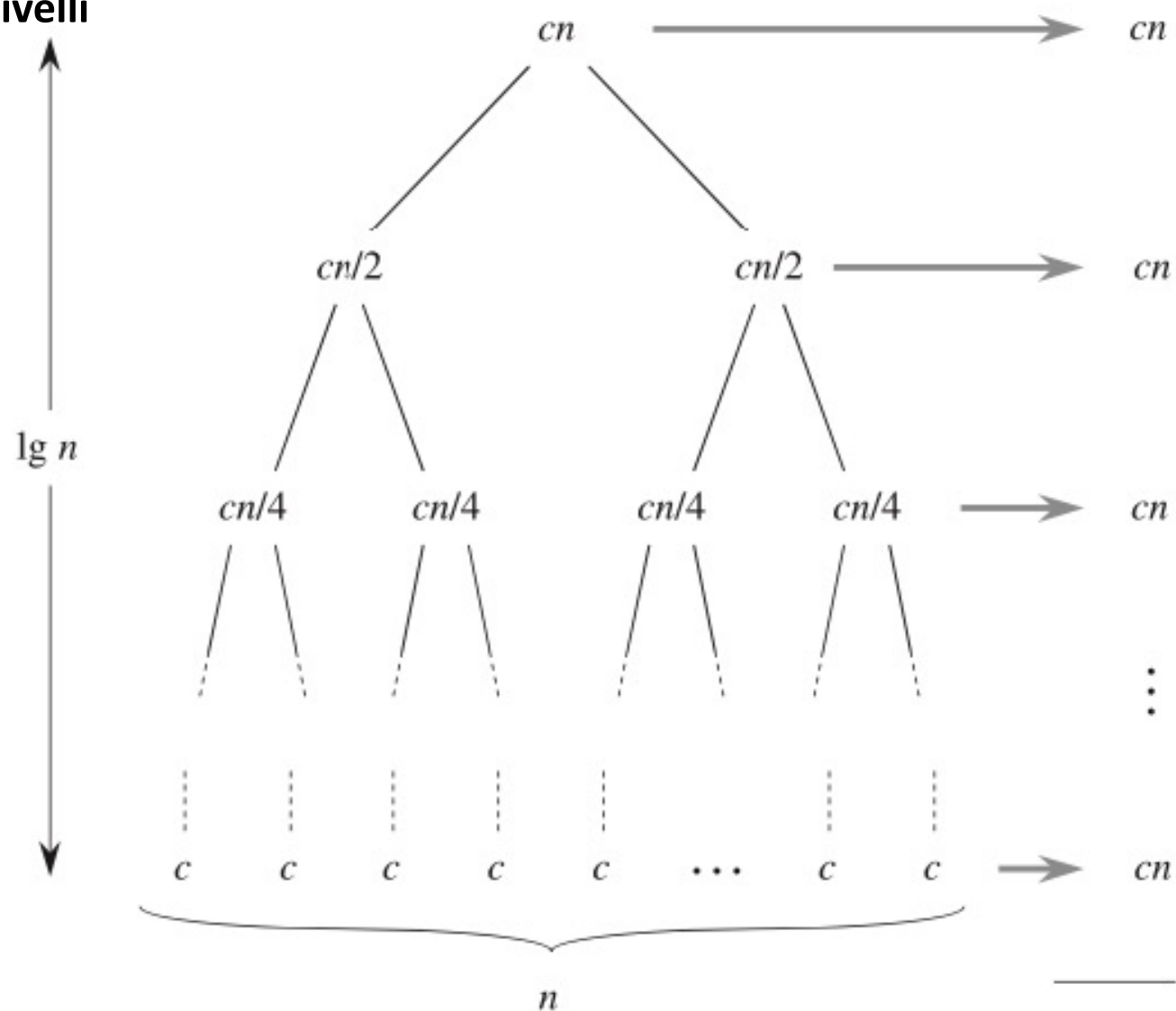
Merge Sort - Complessità

Un albero binario con k livelli
ha $n = 2^{(k-1)}$ foglie

(1 livello: solo root)
(2 livelli: 2^1 foglie)

n foglie $\rightarrow$
 $\lceil \lg_2 n \rceil + 1$ livelli,
ciascuno di costo cn

costo totale
 $cn \lg_2 n + cn$



Merge Sort - Complessità

Relazione tra numero di nodi n e numero di livelli

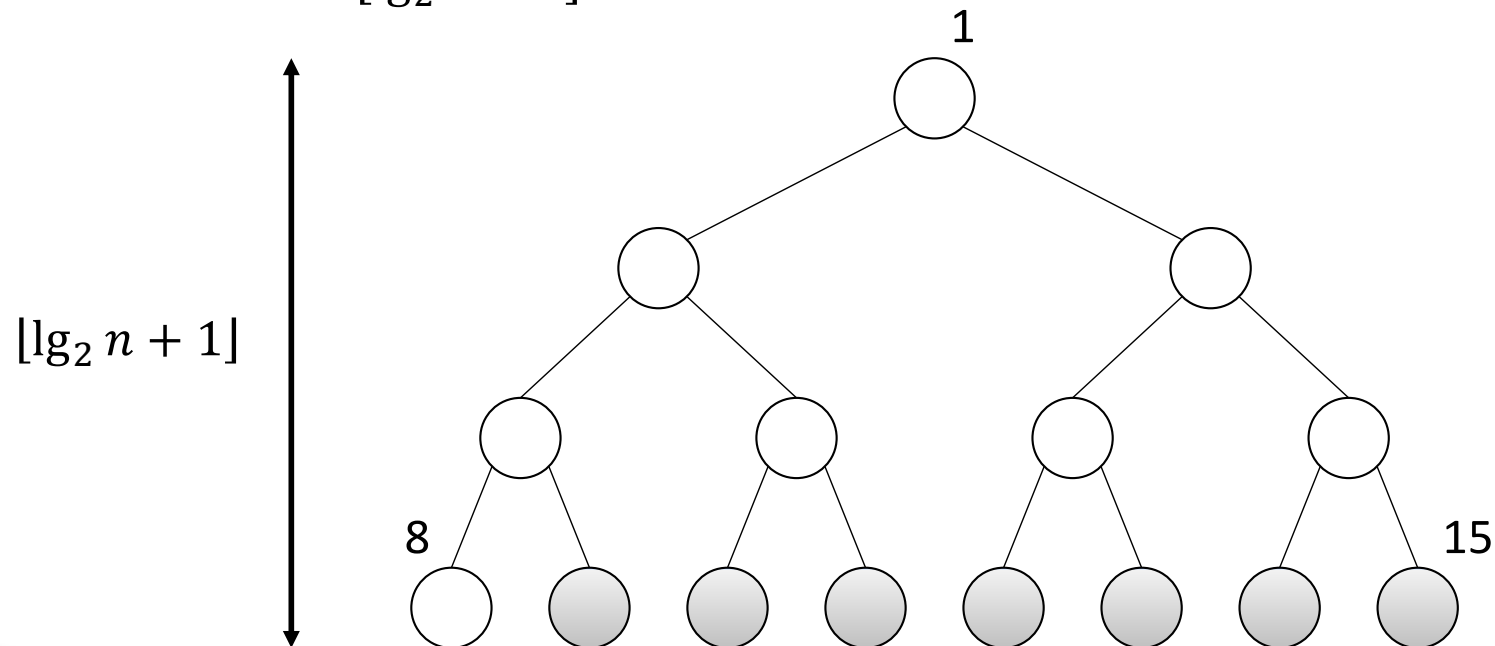
$n \rightarrow \lfloor \lg_2 n + 1 \rfloor$ livelli,

Esempio:

un albero binario con 4 livelli può avere da 8 a 15 nodi

se $n = 8 = 2^3 \rightarrow \lfloor \lg_2 n + 1 \rfloor = 4$

se $n = 15 = 2^4 - 1 \rightarrow \lfloor \lg_2 n + 1 \rfloor = 4$



Esercizi

- Illustrate l'operazione di Merge Sort su un array dato
- Calcolare la complessità $f(n)$ utilizzando come input un array dato. Verificare la correttezza di $f(n)$ contando le istruzioni durante l'esecuzione.
- Modificare Merge Sort in modo che disponga gli elementi in ordine non crescente
- Altri esercizi proposti per gli altri algoritmi di ordinamento

Esercizi

Sebbene merge sort venga eseguito nel tempo $O(n \lg n)$ nel caso peggiore e insertion sort venga eseguito nel tempo $O(n^2)$ nel caso peggiore, **i fattori costanti di insertion sort lo rendono piu' veloce per piccoli valori di n .**

Quindi, ha senso usare insertion sort all'interno di merge sort quando i sottoproblemi diventano sufficientemente piccoli.

Considerate una versione modificata di merge sort in cui n/k sottoliste di lunghezza k siano ordinate utilizzando insertion sort e poi fuse mediante il meccanismo standard di merge sort.

Teorema

La complessità computazionale (tempo) minima di un **qualsiasi** algoritmo di ordinamento per confronto è pari a

$O(n \log n)$